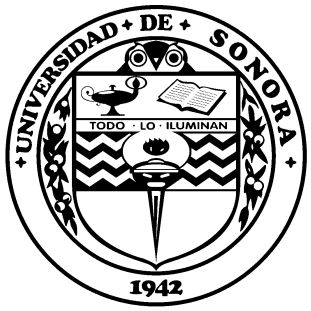


Universidad de Sonora

Licenciatura en Ciencias de la Computación

Investigación sobre detección de errores

Código polinomial y distancia (de Hamming)

Materia: Redes de Computadoras I
Periodo: 2026-2
Profesor: Dr. Donald José Rodríguez Úbeda

Integrantes del equipo

Angel Alberto Amaya Soria	Expediente 223210350
Angel Fernando Bórquez Guerrero	Expediente 219208106
Emily Margarita Beltrán Zazueta	Expediente 223203546
Gabriela Aydee Miranda	Expediente 222220740
Javier Leonardo Miranda Sánchez	Expediente 223215039

Hermosillo, Sonora — Noviembre de 2026

Índice

1. Introducción **2**

2. Conceptos básicos **2**

3. Código polinomial (CRC) **2**
3.1. Cómo funciona 2
3.2. Qué errores detecta 3
3.3. Polinomios generadores de uso común 4
3.4. Ejemplos 4
3.5. Cómo se usa en la comunicación 5

4. Distancia (distancia de Hamming) **6**
4.1. Cómo funciona 6
4.2. Ejemplos 7
4.3. Código de Hamming 7
4.4. Cómo se usa en la comunicación 8

5. Comparación **9**

6. Conclusiones **9**

Referencias **11**

1. Introducción

Cuando los datos viajan por un medio físico (cable de cobre, fibra óptica o el aire), el ruido, la interferencia y la atenuación pueden cambiar algunos bits: un `0` llega como `1` o al revés. La capa de enlace de datos no puede evitar que esto ocurra, pero sí puede *detectarlo*, y en algunos casos corregirlo, agregando a cada mensaje bits redundantes calculados a partir de los datos (Tanenbaum & Wetherall, 2011).

Este reporte describe dos de los conceptos que se emplean para lograrlo: el **código polinomial**, conocido en la práctica como verificación de redundancia cíclica (CRC), y la **distancia** entre palabras de código (distancia de Hamming). Para cada uno se explica cómo funciona, se muestran ejemplos resueltos paso a paso y se describe cómo se utiliza en la comunicación de datos.

2. Conceptos básicos

Un *error de bit* ocurre cuando un solo bit de la trama cambia de valor. Un *error de ráfaga* de longitud b ocurre cuando se altera una secuencia de bits cuyo primer y último bit son erróneos, aunque los de en medio pueden o no haber cambiado (Forouzan, 2013). En la práctica las ráfagas son más comunes que los errores aislados, porque el ruido suele durar más que el tiempo de un solo bit.

Para detectar errores, el emisor transmite una *palabra de código* de $n = m + r$ bits: m bits de datos y r bits de redundancia calculados con una regla conocida por ambos extremos. El receptor aplica la misma regla; si el resultado no es consistente, sabe que hubo un error (Stallings, 2014).

3. Código polinomial (CRC)

3.1. Cómo funciona

Los códigos polinomiales, también llamados códigos de redundancia cíclica, tratan una cadena de bits como los coeficientes de un polinomio con coeficientes `0` y `1`. Una trama de k bits se ve como un polinomio de grado $k-1$ en el que el bit más a la izquierda es el coeficiente de x^{k-1}. Por ejemplo, la cadena `110001` representa $x^5 + x^4 + x^0$ (Tanenbaum & Wetherall, 2011).

La aritmética es módulo 2, sin acarreos: la suma y la resta son ambas la operación XOR, de modo que $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$ y $1 + 1 = 0$. La división larga funciona como la división binaria usual, pero cada resta se sustituye por un XOR.

Emisor y receptor acuerdan de antemano un **polinomio generador** $G(x)$ de grado r, cuyo primer y último coeficiente son 1. Con él, el emisor sigue estos pasos (Peterson & Brown, 1961; Tanenbaum & Wetherall, 2011):

1. Agregar r ceros al final de la trama de datos $M(x)$, lo que equivale a calcular $x^r M(x)$.

2. Dividir (módulo 2) la cadena resultante entre $G(x)$ y obtener el residuo $R(x)$, que tiene a lo más r bits.

3. Restar (XOR) el residuo de $x^r M(x)$. El resultado, $T(x) = x^r M(x) + R(x)$, es la trama que se transmite. En la práctica esto significa colocar el residuo en los r bits finales, y se le llama secuencia de verificación de trama (FCS).

Por construcción, $T(x)$ es divisible entre $G(x)$. El receptor recibe $T'(x)$ y la divide entre $G(x)$: si el residuo es cero, acepta la trama; si es distinto de cero, hubo un error y la descarta (Stallings, 2014).

Si $E(x)$ es el polinomio de error (tiene un 1 en cada posición de bit que se alteró), lo que llega es $T'(x) = T(x) + E(x)$. Como $T(x)$ es divisible entre $G(x)$, el residuo en el receptor es el de $E(x)$ entre $G(x)$. Por lo tanto, **un error pasa sin ser detectado únicamente si $E(x)$ es divisible entre $G(x)$** (Tanenbaum & Wetherall, 2011).

3.2. Qué errores detecta

De esa condición se obtienen propiedades que dependen de la elección de $G(x)$ (Tanenbaum & Wetherall, 2011; Forouzan, 2013):

- **Errores de un solo bit:** $E(x) = x^i$. Si $G(x)$ tiene al menos dos términos (incluido el término x^0), nunca divide a x^i, así que todos se detectan.

- **Errores en número impar de bits:** se detectan todos si $(x + 1)$ es factor de $G(x)$, ya que ningún polinomio con un número impar de términos es divisible entre $x + 1$.

- **Ráfagas de longitud $\leq r$:** se detectan todas. Una ráfaga de longitud exactamente $r + 1$ pasa inadvertida con probabilidad $1/2^{r-1}$, y las más largas con probabilidad $1/2^r$.

- **Dos errores aislados:** se detectan si $G(x)$ no divide a x^k+1 para ningún k menor que la longitud de la trama; eso lo cumplen los polinomios primitivos usados en la práctica.

Con un generador de grado 32 la probabilidad de que una ráfaga larga pase sin detectarse es aproximadamente 2^{-32}, es decir, menos de una en cuatro mil millones.

3.3. Polinomios generadores de uso común

Nombre	**Polinomio $G(x)$**	**Uso típico**
CRC-12	$x^{12} + x^{11} + x^3 + x^2 + x + 1$	Transmisión de caracteres de 6 bits
CRC-16 (CCITT)	$x^{16} + x^{12} + x^5 + 1$	HDLC, X.25, PPP
CRC-32 (IEEE 802.3)	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	Ethernet, Wi-Fi, ZIP, PNG

Tabla 1. Polinomios generadores estándar (Tanenbaum & Wetherall, 2011; IEEE, 2018).

3.4. Ejemplos

Ejemplo 1: cálculo del CRC

Datos: $M =$ `1101011111` ($m = 10$ bits). Generador: $G =$ `10011`, es decir $G(x) = x^4 + x + 1$, de grado $r = 4$.

Paso 1. Se agregan 4 ceros: `11010111110000`.

Paso 2. División módulo 2 (cada línea toma una ventana de 5 bits que empieza en un 1, le hace XOR con `10011` y los bits siguientes bajan; las posiciones donde el bit de la izquierda es 0 simplemente se saltan):

```
11010111110000 : 10011
11010  xor 10011 -> 01001
 10011 xor 10011 -> 00000
      11110 xor 10011 -> 01101
       11010 xor 10011 -> 01001
        10010 xor 10011 -> 00001
Residuo (ultimos 4 bits) = 0010
```

Paso 3. La trama transmitida es los datos seguidos del residuo:

$$T = \texttt{1101011111}\,\texttt{0010} = \texttt{11010111110010}.$$

Verificación en el receptor sin errores. Al dividir `11010111110010` entre `10011` el residuo es `0000`, de modo que la trama se acepta.

Con errores. Los siguientes resultados se obtuvieron dividiendo la trama alterada entre el mismo generador:

Caso	Trama recibida	Residuo	Resultado
Sin error	11010111110010	0000	Se acepta
Error de 1 bit (3.er bit)	11110111110010	1110	Se detecta
Ráfaga de 3 bits (bits 4 al 6)	11001011110010	1000	Se detecta
Dos errores (primer y último bit)	01010111110011	1100	Se detecta
$E =$ 00000010011000	11010101101010	0000	**No se detecta**

Tabla 2. Detección de errores sobre la trama del ejemplo 1.

El último caso ilustra la limitación del método. El patrón de error 00000010011000 es exactamente $G(x)$ desplazado tres posiciones, o sea $E(x) = x^3 G(x)$, que es divisible entre $G(x)$. Es una ráfaga de longitud $5 = r + 1$, el caso en que existe una pequeña probabilidad de que el error pase inadvertido.

Ejemplo 2: ejemplo pequeño

Datos: $M =$ 100100, generador $G =$ 1101 ($x^3 + x^2 + 1$, $r = 3$). Se divide 100100000 entre 1101:

```
100100000 : 1101
1001  xor 1101 -> 0100
 1000 xor 1101 -> 0101
  1010 xor 1101 -> 0111
   1110 xor 1101 -> 0011
     1100 xor 1101 -> 0001
Residuo (ultimos 3 bits) = 001
```

La trama transmitida es $T =$ 100100 001 $=$ 100100001. Al dividirla en el receptor se obtiene residuo 000, así que se acepta.

3.5. Cómo se usa en la comunicación

El CRC es el mecanismo de detección de errores dominante en la capa de enlace de datos, por varias razones: detecta muy bien las ráfagas, agrega pocos bits (típicamente 16 o 32) y es sencillo de implementar en hardware con un registro de desplazamiento realimentado con compuertas XOR (Stallings, 2014; Tanenbaum & Wetherall, 2011).

El uso típico es el siguiente (Kurose & Ross, 2017):

1. El adaptador de red del emisor calcula el CRC sobre los bytes de la trama y lo coloca al final, en el campo FCS.

2. El adaptador del receptor recalcula el CRC sobre lo recibido. Si el resultado no es cero, la trama se descarta sin entregarla al nivel superior.

3. La recuperación depende del protocolo: en Ethernet la trama dañada simplemente se descarta y, si hace falta, protocolos de capas superiores como TCP la retransmiten; en otros enlaces (por ejemplo, Wi-Fi o HDLC) el propio enlace solicita la retransmisión con un esquema ARQ.

Ethernet usa un CRC de 32 bits como FCS en cada trama (IEEE, 2018), y el mismo polinomio se emplea en IEEE 802.11, ZIP y PNG, mientras que PPP y HDLC usan CRC-16 o CRC-32 (Tanenbaum & Wetherall, 2011). Cabe señalar que el CRC está diseñado para errores aleatorios del canal, no para detectar modificaciones malintencionadas, ya que es fácil construir datos distintos con el mismo CRC. Para eso se usan funciones hash criptográficas. La elección del polinomio también influye mucho en la capacidad real de detección según la longitud del mensaje (Koopman & Chakravarty, 2004).

4. Distancia (distancia de Hamming)

4.1. Cómo funciona

La **distancia de Hamming** entre dos cadenas de bits de la misma longitud es el número de posiciones en que difieren. Se calcula con un XOR bit a bit y contando los unos del resultado (Hamming, 1950; Tanenbaum & Wetherall, 2011).

Al usar m bits de datos y r de verificación, solo 2^m de las 2^n palabras posibles ($n = m+r$) son válidas. A ese conjunto de palabras válidas se le llama *código*. La **distancia mínima** $d_{\text{mín}}$ del código es la menor distancia de Hamming entre dos palabras válidas distintas, y determina sus capacidades (Hamming, 1950; Stallings, 2014):

- Para **detectar** hasta t errores se necesita $d_{\text{mín}} \geq t+1$, ya que t errores no pueden convertir una palabra válida en otra palabra válida.
- Para **corregir** hasta t errores se necesita $d_{\text{mín}} \geq 2t+1$: con ello, la palabra recibida sigue estando más cerca de la palabra original que de cualquier otra, y el receptor la decodifica a la palabra válida más cercana.

$d_{\text{mín}}$	**Errores que detecta** ($d_{\text{mín}} - 1$)	**Errores que corrige** ($\lfloor (d_{\text{mín}} - 1)/2 \rfloor$)
1	0	0
2	1	0
3	2	1
4	3	1
5	4	2

Tabla 3. Capacidad de un código según su distancia mínima.

4.2. Ejemplos

Ejemplo 3: distancia entre dos palabras

Sean `10110` y `10011`. Su XOR es `00101`, que tiene dos unos, por lo que $d(\texttt{10110}, \texttt{10011}) = 2$.

Ejemplo 4: bit de paridad

Si a cada palabra de datos se le agrega un bit de paridad par, dos palabras válidas distintas siempre difieren en al menos dos bits (si cambia un bit de datos también debe cambiar la paridad). Entonces $d_{\text{mín}} = 2$: detecta todos los errores de un bit, pero no puede decir cuál bit cambió, por lo que no corrige. Tampoco detecta errores de un número par de bits.

Ejemplo 5: código con distancia 5

Considere el código de cuatro palabras válidas

$$\{\texttt{0000000000},\ \texttt{0000011111},\ \texttt{1111100000},\ \texttt{1111111111}\}.$$

Las distancias entre pares son 5 o 10, por lo que $d_{\text{mín}} = 5$: detecta hasta 4 errores y corrige hasta 2 (Tanenbaum & Wetherall, 2011). Si se recibe `0000000111`, las distancias a las palabras válidas son 3 (a `0000000000`), 2 (a `0000011111`), 7 y 8, así que el receptor decodifica `0000011111`. Si en cambio hubiera tres errores en `0000000000` y se recibiera `0000000111`, el receptor se equivocaría al "corregir", ya que se excedió su capacidad de corrección.

4.3. Código de Hamming

Hamming (1950) propuso un método para construir códigos de distancia 3 que corrigen un error de bit con pocos bits de redundancia. Los bits de verificación ocupan las posiciones que son potencias de 2 (1, 2, 4, 8, ...), numeradas desde la izquierda, y los bits de datos ocupan las demás. La posición p de un bit se escribe en binario, y cada bit de verificación se calcula con la paridad par de todas las posiciones cuya representación binaria incluye esa potencia de 2 (Tanenbaum & Wetherall, 2011). Por ejemplo, el bit 1 verifica las posiciones 1, 3, 5, 7, 9, 11, ...; el bit 2 verifica 2, 3, 6, 7, 10, 11, ...; el bit 4 verifica 4, 5, 6, 7, 12, ...

Para corregir un error de un bit entre m bits de datos se necesitan r bits de verificación que cumplan

$$m + r + 1 \leq 2^r.$$

En el receptor se recalcula cada verificación y se suman las posiciones de las verificaciones que fallan: ese número, llamado *síndrome*, es justamente la posición del bit erróneo (si es 0, no hay error), y se corrige invirtiéndolo.

Ejemplo 6: Hamming (7,4)

Datos: `1011`. Con $r = 3$ ($4 + 3 + 1 = 8 \leq 2^3$), los datos ocupan las posiciones 3, 5, 6 y 7 y las verificaciones las posiciones 1, 2 y 4:

Posición	1	2	3	4	5	6	7
Tipo	p_1	p_2	d	p_4	d	d	d
Dato			1		0	1	1
Palabra	0	1	1	0	0	1	1

- p_1 verifica 1, 3, 5, 7: $1 \oplus 0 \oplus 1 = 0$, por lo que $p_1 = 0$.
- p_2 verifica 2, 3, 6, 7: $1 \oplus 1 \oplus 1 = 1$, por lo que $p_2 = 1$.
- p_4 verifica 4, 5, 6, 7: $0 \oplus 1 \oplus 1 = 0$, por lo que $p_4 = 0$.

La palabra transmitida es `0110011`.

Error en el bit 5. Se recibe `0110111`. Fallan las verificaciones de p_1 (posiciones 1, 3, 5, 7: $0 \oplus 1 \oplus 1 \oplus 1 = 1$) y de p_4 (4, 5, 6, 7: $0 \oplus 1 \oplus 1 \oplus 1 = 1$), mientras que p_2 pasa. El síndrome es $1 + 4 = 5$, así que el bit 5 es el erróneo, se invierte y se recupera `0110011`.

Dos errores. Si se alteran los bits 2 y 5 se recibe `0010111`, con síndrome $2 \oplus 5 = 7$. El receptor "corregiría" el bit 7 y entregaría datos incorrectos. Por eso el código de Hamming simple, con $d_{\text{mín}} = 3$, corrige un error o detecta dos, pero no ambas cosas a la vez. Agregando un bit de paridad global se obtiene un código de distancia 4 que corrige un error y detecta dos (SECDED) (Stallings, 2014).

Ejemplo 7: carácter ASCII

Para la letra "H" (`1001000`, $m = 7$) se requieren $r = 4$ bits de verificación ($7 + 4 + 1 = 12 \leq 16$), con lo que $n = 11$. Colocando los datos en las posiciones 3, 5, 6, 7, 9, 10 y 11 y calculando p_1, p_2, p_4 y p_8 con paridad par, se obtiene la palabra `00110010000`. Si durante la transmisión se invierte la posición 6 y se recibe `00110110000`, fallan las verificaciones de p_2 y p_4, de modo que el síndrome es $2 + 4 = 6$ y se corrige el bit 6 (Tanenbaum & Wetherall, 2011).

4.4. Cómo se usa en la comunicación

Existen dos estrategias para tratar los errores (Tanenbaum & Wetherall, 2011):

- **Detección con retransmisión (ARQ).** Se agregan pocos bits de redundancia (código de $d_{\text{mín}}$ pequeña, como CRC o paridad), el receptor detecta el error y pide el reenvío. Es la opción habitual en canales confiables como el cable de cobre o la fibra, donde los errores son raros.
- **Corrección hacia adelante (FEC).** Se agrega más redundancia (código con $d_{\text{mín}} \geq 3$, como Hamming) para que el receptor corrija por sí mismo. Se prefiere en canales ruidosos, o cuando retransmitir es caro o imposible, como los enlaces inalámbricos, los satelitales y los de larga distancia, o en el almacenamiento y la memoria (Stallings, 2014).

Los códigos tipo Hamming con SECDED se usan en la memoria con corrección de errores (ECC) de servidores, que corrige un bit y detecta dos por palabra. En comunicaciones modernas se emplean códigos con mayor distancia y capacidad (Reed-Solomon, convolucionales, LDPC), pero los conceptos de distancia mínima y de decodificación a la palabra más cercana que introdujo Hamming siguen siendo su fundamento (Hamming, 1950; Stallings, 2014).

5. Comparación

	Código polinomial (CRC)	**Distancia / código de Hamming**
Idea central	Residuo de la división polinomial módulo 2 entre un generador $G(x)$	Separación mínima $d_{\text{mín}}$ entre palabras válidas; paridades sobre posiciones
Redundancia	r bits fijos (16 o 32), sin importar el tamaño de la trama	r crece con m ($m + r + 1 \leq 2^r$)
Fortaleza	Detecta ráfagas y la mayoría de los errores	Corrige errores de 1 bit sin retransmitir
Limitación	Solo detecta; no localiza el error	Un patrón de 2 errores puede corregirse mal
Uso típico	FCS en Ethernet, Wi-Fi, PPP, HDLC	Memoria ECC, FEC en canales ruidosos

Tabla 4. Comparación entre ambos métodos.

6. Conclusiones

El código polinomial convierte una trama en un polinomio y transmite el residuo de dividirlo entre un generador común; el receptor solo debe verificar que el residuo de la trama completa sea cero. Con pocos bits detecta casi todos los errores, incluidas

ráfagas largas, y por eso es el mecanismo estándar en la capa de enlace. La distancia de Hamming, por su parte, ofrece el criterio para diseñar códigos: $d_{\text{mín}}$ determina cuántos errores se detectan ($d_{\text{mín}} - 1$) y cuántos se corrigen ($\lfloor (d_{\text{mín}} - 1)/2 \rfloor$). Los dos enfoques son complementarios: el CRC es económico y sirve para detectar y pedir retransmisión, mientras que los códigos de Hamming gastan más redundancia pero permiten corregir sin retransmitir. Elegir entre ellos depende de la tasa de errores del canal y del costo de retransmitir.

Referencias

Forouzan, B. A. (2013). *Data communications and networking* (5.ª ed.). McGraw-Hill.

Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal, 29*(2), 147–160.

Institute of Electrical and Electronics Engineers. (2018). *IEEE Std 802.3-2018: IEEE standard for Ethernet*. IEEE.

Koopman, P., & Chakravarty, T. (2004). Cyclic redundancy code (CRC) polynomial selection for embedded networks. En *Proceedings of the International Conference on Dependable Systems and Networks (DSN 2004)* (pp. 145–154). IEEE.

Kurose, J. F., & Ross, K. W. (2017). *Computer networking: A top-down approach* (7.ª ed.). Pearson.

Peterson, W. W., & Brown, D. T. (1961). Cyclic codes for error detection. *Proceedings of the IRE, 49*(1), 228–235.

Stallings, W. (2014). *Data and computer communications* (10.ª ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer networks* (5.ª ed.). Prentice Hall.